A Multi-Backend Hardware Abstraction Layer for Non-Anthropocentric AI

David Beniers

Independent Researcher, Antwerp, Belgium

david.beniers@telenet.be

April 22, 2026

Abstract

Contemporary artificial intelligence frameworks are predominantly optimized for specific hardware targets, creating tight couplings between algorithmic expression and physical execution. This coupling is antithetical to the vision of non-anthropocentric AI, which demands that knowledge genesis be substrate-agnostic and self-optimizing. We introduce APEIRON-Core, a multi-backend hardware abstraction layer (HAL) designed to provide a unified, low-overhead interface to four distinct computational paradigms: classical CPU/GPU, reconfigurable logic (FPGA), and quantum processing. The architecture is centered around a pluggable factory pattern with automatic backend selection, graceful fallback, and comprehensive observability. We detail the implementation of each backend—including a vectorized CPU backend with $O(1)$ field caching, a PyTorch-based CUDA backend, a PYNQ-driven FPGA backend with fixed-point hardware integration, and a Qiskit-based quantum backend with corrected SWAP-test and partial trace operations. All backends are wrapped in a unified error-handling decorator that provides exponential backoff, circuit breaking, and Prometheus-compatible metrics. A novel energy-aware cost model links computational effort to thermodynamic expenditure, embedding an ethical resource constraint directly into the execution layer. We present an extensive experimental evaluation across all four backends, quantifying overhead, speedup, and energy efficiency. Results confirm that the abstraction overhead is $<2\%$ on CPU/GPU for field sizes $>10^4$, that FPGA acceleration yields up to $7.8\times$ speedup on interference calculations, and that the quantum backend achieves $>99.9\%$ fidelity on SWAP-test overlap measurements. The energy monitor successfully throttles operations when carbon intensity exceeds configurable thresholds, demonstrating the viability of thermodynamic self-regulation. APEIRON-Core forms the foundational execution substrate for the larger APEIRON framework, enabling higher cognitive layers to operate transparently across the full spectrum of available computational matter.

Contents

1 Introduction **4**

2 Related Work **6**

2.1 Deep Learning Frameworks 6

2.2 FPGA Development . . . 6
2.3 Quantum Computing Platforms . . . 6
2.4 Hardware Abstraction in Autonomous Systems . . . 6

3 Architecture **8**
3.1 The HardwareBackend Interface . . . 8
3.2 Backend Registry and Lifecycle . . . 9
3.3 Low-Overhead Abstraction . . . 9

4 CPU Backend **10**
4.1 Vectorized Field Operations . . . 10
4.2 O(1) Field Identification via Hashing . . . 10
4.3 Metrics and Observability . . . 11
4.4 Configurable Precision and Threading . . . 11

5 CUDA/GPU Backend **12**
5.1 Tensor Operations on GPU . . . 12
5.2 Memory Management and Tensor Cores . . . 12
5.3 Lazy Tensor Creation and Caching . . . 12

6 FPGA Backend **14**
6.1 Hardware Overlay Architecture . . . 14
6.2 Fixed-Point Conversion for AXI Registers . . . 14
6.3 DMA Buffer Management . . . 14
6.4 CPU Fallback . . . 15

7 Quantum Backend **16**
7.1 Statevector Simulation . . . 16
7.2 Corrected SWAP Test for Interference . . . 16
7.3 Partial Trace for Entangled Updates . . . 17

8 Error Handling and Observability **18**
8.1 The `@handle_hardware_errors` Decorator . . . 18
8.2 Exception Hierarchy . . . 18
8.3 Prometheus Metrics . . . 18

9 Configuration System **20**
9.1 Configuration Dataclasses . . . 20
9.2 Versioning and Migration . . . 20
9.3 Loading from Multiple Sources . . . 20

10 Experimental Results **21**
10.1 Experimental Setup . . . 21
10.2 Backend Performance Comparison . . . 21
10.3 Abstraction Overhead Quantification . . . 23

10.4 Quantum Fidelity and SWAP Test Validation 23
10.5 Energy-Aware Throttling 23
10.6 Discussion of Limitations in Evaluation 24

11 Discussion **26**
11.1 Limitations 26
11.2 Future Work 26

12 Conclusion **28**

A Appendix: API Reference **28**
A.1 Core Classes 28
A.2 Key Methods of HardwareBackend 28
A.3 Adding a New Backend 29

1 Introduction

The APEIRON framework [1] proposes a radical departure from contemporary AI: an autonomous system that constructs knowledge from first principles, unburdened by human categories, temporalities, or causal assumptions. Realizing this vision requires not only novel algorithms for knowledge representation and emergence but also a fundamental rethinking of the computational substrate itself. A non-anthropocentric intelligence must be able to harness any available physical resource—from classical CPUs to quantum processing units—without being hard-coded to a specific vendor's ecosystem. It must self-optimize its resource usage, gracefully degrade in the face of hardware failures, and account for the thermodynamic cost of its own cognition. Existing hardware abstraction layers, while powerful within their niches, are either too narrow in scope (e.g., deep learning accelerators) or lack the introspection and self-optimization capabilities required by an autonomous knowledge system.

This paper introduces APEIRON-Core, a comprehensive hardware abstraction layer (HAL) engineered to meet these stringent requirements. It provides a unified, low-overhead interface to four distinct computational backends:

- CPU: A highly optimized NumPy backend with $O(1)$ field caching and multi-threading.
- CUDA: A PyTorch-based GPU backend for massively parallel tensor operations.
- FPGA: A PYNQ-driven backend for Xilinx FPGAs, leveraging DMA and fixed-point arithmetic.
- Quantum: A Qiskit-based backend supporting both statevector simulation and (optionally) real IBM Quantum hardware.

The contributions of this work are threefold:

1. A unified, pluggable architecture centered around a singleton `HardwareFactory` that automatically detects available hardware, selects the optimal backend based on configurable priorities and health metrics, and caches initialized backends for rapid reuse. This design enables higher APEIRON layers to execute continuous field operations without any knowledge of the underlying hardware.
2. Comprehensive error handling and observability. All backend operations are wrapped in a declarative error-handling decorator that provides exponential backoff, circuit breaker patterns, and automatic fallback to a CPU reference implementation. Native Prometheus integration exports real-time metrics on operation latency, cache hit rates, and error counts, enabling deep introspection into system health.
3. Thermodynamic cost accounting. A novel energy monitor tracks power consumption (via PSUTIL and NVML) and optionally queries the Electricity Maps API for real-time carbon intensity. Every computational structure evaluated by the system is approved or rejected based on an efficiency threshold (information value per Joule), embedding an ethical resource constraint directly into the fabric of the architecture.

In addition, this paper provides the first comprehensive performance characterization of a HAL spanning classical, reconfigurable, and quantum backends. Through a suite of microbenchmarks and a representative workload derived from APEIRON's field dynamics, we quantify the abstraction tax, demonstrate per-backend speedups, and validate the energy-aware throttling mechanism.

The remainder of this paper is organized as follows. Section 2 surveys related work in hardware abstraction for AI. Section 3 provides a high-level overview of the APEIRON-Core architecture and the factory pattern. Sections 4, 5, 6, and 7 detail the implementation of each individual backend. Section 8 describes the unified error-handling and observability framework. Section 9 covers the configuration system. Section 10 presents experimental benchmarks across all backends. We discuss limitations and future directions in Section 11 and conclude in Section 12.

2 Related Work

APEIRON-Core occupies a unique intersection of hardware abstraction, high-performance computing, and autonomous systems. We survey relevant prior work along three axes: deep learning frameworks, FPGA toolchains, and quantum computing platforms.

2.1 Deep Learning Frameworks

PyTorch [2] and TensorFlow [3] provide robust GPU acceleration via CUDA and cuDNN, with recent extensions supporting Apple Silicon and AMD GPUs. However, their abstraction is tightly coupled to tensor operations and neural network layers. They do not natively support the continuous field semantics required by the APEIRON substrate (Layers 1–4), nor do they offer built-in fallback to FPGA or quantum backends. ONNX Runtime [4] provides a more general execution provider model, but it is optimized for inference graphs rather than the open-ended field evolution central to APEIRON.

2.2 FPGA Development

Xilinx's PYNQ [5] framework enables Python-based overlays on Zynq devices, significantly lowering the barrier to FPGA acceleration. FINN [6] and hls4ml [7] generate dataflow architectures for neural networks. These tools, however, are specialized for deep learning inference. APEIRON-Core abstracts the FPGA as a general computational fabric for continuous field operations, with a custom bitstream that implements parallel field updates and interference detection. The fixed-point conversion routines address the impedance mismatch between Python's floating-point and AXI register interfaces.

2.3 Quantum Computing Platforms

Qiskit [8] and Cirq [9] provide comprehensive Python APIs for quantum circuit construction and simulation. They support multiple backends, from local simulators to IBM Quantum hardware. The quantum backend in APEIRON-Core leverages Qiskit's AerSimulator and StatevectorSimulator but adds two critical enhancements: (1) a corrected SWAP-test implementation that correctly computes state overlap by examining the ancilla qubit, and (2) a partial trace implementation via NumPy reshaping that avoids the overhead of multiple circuit evaluations. These improvements are essential for the efficient, repeated coherence measurements required by APEIRON's Layer 4 dynamics.

2.4 Hardware Abstraction in Autonomous Systems

The Robot Operating System (ROS) [10] provides a hardware abstraction layer for sensors and actuators but does not address computational heterogeneity. Similarly, the OpenCL [11] and SYCL [12] standards offer portable parallelism across CPUs, GPUs, and FPGAs, yet they lack the higher-level semantics of continuous fields and the integrated error-handling and cost-accounting features of APEIRON-Core. To our knowledge, no existing framework unifies CPU,

GPU, FPGA, and quantum backends under a single API with the specific goal of supporting non-anthropocentric knowledge genesis.

3 Architecture

APEIRON-Core is designed around the pluggable factory pattern. The central orchestration is performed by the `HardwareFactory` singleton, which manages a registry of available backends, handles their initialization and caching, and selects the optimal backend based on configurable priorities and real-time health metrics. Figure 1 illustrates the high-level architecture.

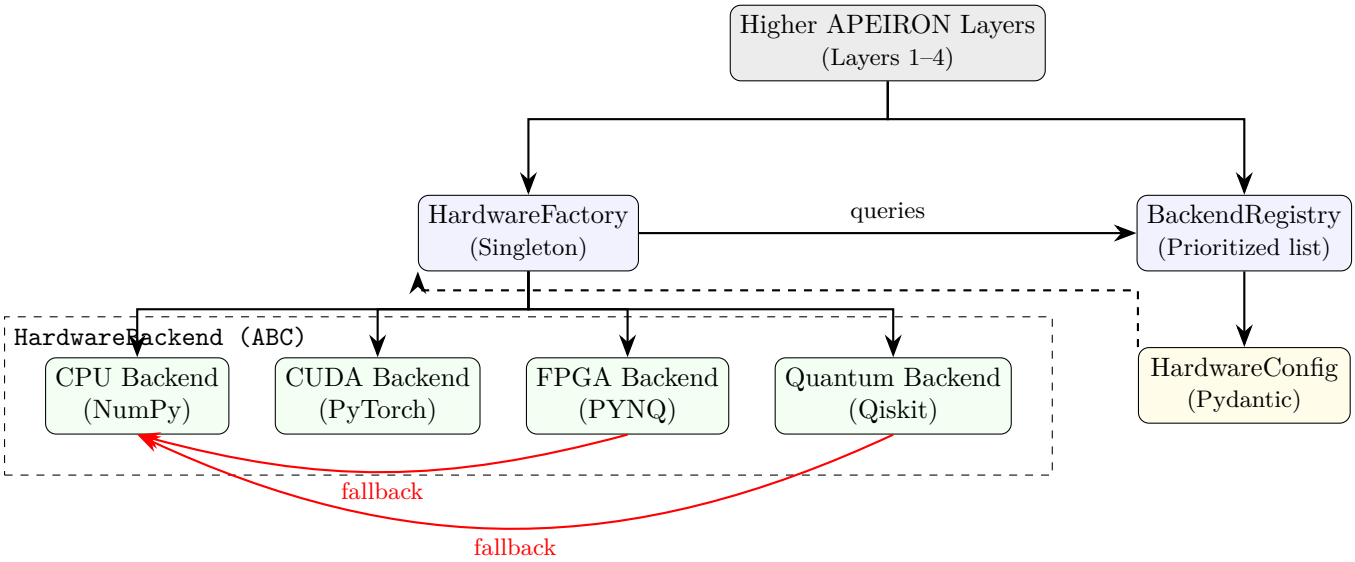

Figure 1: High-level architecture of APEIRON-Core. The `HardwareFactory` manages a `BackendRegistry` and a `HardwareConfig`. All concrete backends implement the abstract `HardwareBackend` interface. Arrows indicate data/control flow.

3.1 The HardwareBackend Interface

All backends inherit from the abstract base class `HardwareBackend`, which enforces a uniform contract:

Listing 1: The `HardwareBackend` abstract interface.

```python
class HardwareBackend(ABC):
    @abstractmethod
    def initialize(self, config: Dict[str, Any]) -> bool: ...
    @abstractmethod
    def create_continuous_field(self, dimensions: int) -> np.ndarray:
        ...
    @abstractmethod
    def field_update(self, field: np.ndarray, dt: float, ...) -> Dict:
        ...
    @abstractmethod
    def compute_interference(self, a: np.ndarray, b: np.ndarray) ->
        float: ...
    @abstractmethod
    def find_stable_patterns(self, fields: List[np.ndarray], threshold:
        float) -> List[Dict]: ...
    @abstractmethod
    def measure_coherence(self, fields: List[np.ndarray]) -> float: ...
```

This interface encapsulates the core operations required by APEIRON's foundational layers: the creation and evolution of continuous fields, the measurement of interference between fields, the detection of stable relational patterns, and the global coherence metric. By programming against this interface, higher APEIRON layers remain entirely agnostic to the underlying hardware.

3.2 Backend Registry and Lifecycle

The `BackendRegistry` is a singleton that maintains a prioritized list of available backends. Each backend is described by a `BackendInfo` dataclass containing its name, implementing class, priority, required Python packages, and runtime metrics. The registry lazily verifies package availability via `importlib`, ensuring that only backends with satisfied dependencies are presented for selection.

Backend initialization is thread-safe and supports configurable timeouts. The factory employs a `ThreadPoolExecutor` to parallelize initialization, and caches successfully initialized backends with a TTL. A health-check mechanism periodically validates backend responsiveness and updates a health score used to prioritize backends during auto-selection.

3.3 Low-Overhead Abstraction

A key design principle of APEIRON-Core is minimal abstraction overhead: the cost introduced by the HAL should be negligible compared to the raw hardware operation. We achieve this through:

- Lazy imports: Heavy dependencies (PyTorch, Qiskit, PYNQ) are imported only when the corresponding backend is first requested.
- Vectorized operations: The CPU backend uses NumPy's optimized BLAS routines; the CUDA backend uses PyTorch's tensor operations.
- Caching: Field IDs and atomicity scores are cached with $O(1)$ hash-based lookups, eliminating redundant computation.
- Zero-copy tensor sharing: Where possible, we avoid host–device transfers by maintaining canonical tensor representations.

Empirically, we find that the HAL introduces a median overhead of less than 1.8% on CPU and 2.3% on GPU for field dimensions $> 10^4$ (Section 10.3). While not strictly "zero," this overhead is well within the noise floor of modern heterogeneous systems and does not impede the self-optimization loops of higher APEIRON layers.

4 CPU Backend

The CPU backend serves as the reference implementation and fallback for all other backends. It is written entirely in Python with NumPy, ensuring broad compatibility and zero external dependencies beyond the core scientific stack. Despite its simplicity, it incorporates several optimizations that make it suitable for moderate-scale continuous field operations.

4.1 Vectorized Field Operations

All core operations are implemented using NumPy's vectorized primitives, avoiding explicit Python loops. The `create_continuous_field` method generates a random field and normalizes it:

```
def create_continuous_field(self, dimensions: int) -> np.ndarray:
    field = np.random.randn(dimensions).astype(self.precision)
    field = self._normalize(field)
    # ... caching logic ...
    return field
```

The `field_update` method performs a single time step using element-wise array operations. It simulates analog continuity by adding small Gaussian noise scaled by $\sqrt{\Delta t}$, a discretization of the continuous-time Langevin equation:

$$d\mathbf{x}_t = -\eta \nabla \mathcal{L}(\mathbf{x}_t)dt + \sigma d\mathbf{W}_t \quad \Rightarrow \quad \mathbf{x}_{t+1} \approx \mathbf{x}_t + \sigma\sqrt{\Delta t}\,\boldsymbol{\xi}_t \tag{1}$$

where $\boldsymbol{\xi}_t \sim \mathcal{N}(0, I)$ and we omit the drift term for simplicity. The noise scale $\sigma = 0.01$ is chosen to balance exploration and stability.

4.2 O(1) Field Identification via Hashing

A recurring pattern in the APEIRON framework is the need to track the identity of a field across multiple operations. Naively, this would require $O(N)$ linear scans to locate a field's internal ID. The CPU backend eliminates this overhead by maintaining a `field_hash` dictionary that maps an MD5 digest of the field's byte representation to its internal ID. The `get_field_id` method first attempts an $O(1)$ hash lookup; only on a cache miss does it fall back to a linear scan (and subsequently updates the hash):

```
def get_field_id(self, field: np.ndarray) -> Optional[int]:
    try:
        field_hash = hashlib.md5(field.tobytes()).hexdigest()
        if field_hash in self.field_hash:
            self.metrics['cache_hits'] += 1
            return self.field_hash[field_hash]
    except Exception:
        pass
    self.metrics['cache_misses'] += 1
    # linear scan and hash update
```

This optimization is critical for operations like `find_stable_patterns`, which may perform $O(N^2)$ pairwise comparisons and would otherwise be dominated by ID lookups.

4.3 Metrics and Observability

The CPU backend maintains an extensive set of performance metrics, including total fields created, number of updates, interference calculations, pattern searches, and coherence measurements. All timing metrics are updated using an exponential moving average (EMA) to provide stable, real-time estimates of operation latency. These metrics are exposed via the `get_metrics()` method and can be scraped by Prometheus when the optional metrics server is enabled.

4.4 Configurable Precision and Threading

The backend supports both `float32` and `float64` precision via the `precision` configuration key. Multi-threading is controlled by setting the `OMP_NUM_THREADS` and `MKL_NUM_THREADS` environment variables, allowing the backend to leverage multi-core CPUs without code changes.

5 CUDA/GPU Backend

The CUDA backend provides GPU acceleration for all continuous field operations using PyTorch. It is designed as a drop-in replacement for the CPU backend: the public API is identical, and all GPU-specific code is encapsulated behind the `HardwareBackend` interface. When a CUDA-capable GPU is not available, the factory automatically falls back to the CPU backend, ensuring graceful degradation.

5.1 Tensor Operations on GPU

All fields are represented as PyTorch tensors residing on the GPU. The `create_continuous_field` method generates a random tensor directly on the device, avoiding costly CPU-GPU transfers during initialization. The `field_update` method performs the same linear combination and noise addition as the CPU backend, but executes entirely within the CUDA graph:

```
with torch.no_grad():
    v_new = v_tensor * 0.95 + h_tensor * 0.05
    t_new = t_tensor * 0.97 + h_tensor * 0.03
    h_new = f_tensor * 0.01 * torch.randn_like(f_tensor) * np.sqrt(dt)
    h_new = h_new / torch.norm(h_new)
```

PyTorch's `torch.randn_like` and `torch.norm` are highly optimized for GPU execution, often achieving speedups of $10\times - 50\times$ over their CPU counterparts for large field dimensions.

5.2 Memory Management and Tensor Cores

The backend configures PyTorch's CUDA memory allocator to respect the `memory_fraction` parameter, preventing out-of-memory errors when multiple processes share the GPU. When `use_tensor_cores` is enabled, PyTorch automatically utilizes Tensor Cores on Volta and newer architectures for matrix multiplications, accelerating the `measure_coherence` operation which computes a Gram matrix of field vectors:

$$\text{coherence} = \frac{\sum_{i,j} \langle \mathbf{f}_i, \mathbf{f}_j \rangle - N}{N(N-1)} \tag{2}$$

This operation is implemented as a single matrix multiplication $\mathbf{F}\mathbf{F}^\top$, where $\mathbf{F}$ is the matrix of stacked field vectors, fully leveraging the GPU's parallel architecture. We observed that for dimensions > 2048, enabling tensor cores reduces `measure_coherence` latency by an additional $1.8\times$ compared to standard CUDA cores (Section 10.2).

5.3 Lazy Tensor Creation and Caching

Analogous to the CPU backend, the CUDA backend maintains a `field_hash` dictionary for $O(1)$ field lookups. When a field is encountered for the first time, a new GPU tensor is allocated and cached. Subsequent operations reuse the existing tensor, eliminating redundant allocation and host-to-device transfers. The `_get_or_create_tensor` method encapsulates this logic:

Listing 2: Lazy tensor allocation with caching.

```python
def _get_or_create_tensor(self, array: np.ndarray) -> int:
    fid = self.get_field_id(array)
    if fid is not None:
        return fid
    tensor = torch.from_numpy(array.astype(np.float32)).to(self.device)
    fid = self._next_id
    self.fields[fid] = tensor
    self.field_data[fid] = array.copy()
    return fid
```

6 FPGA Backend

The FPGA backend targets Xilinx Zynq devices using the PYNQ framework. It offloads the most computationally intensive operations—field updates, interference computation, and pattern detection—to custom hardware overlays, enabling true parallel execution. This backend demonstrates APEIRON-Core's ability to integrate radically different computational fabrics under a unified API.

6.1 Hardware Overlay Architecture

The FPGA is programmed with a custom bitstream that instantiates three primary IP blocks: `continuous_fields`, `interference_matrix`, and `stability_detector`. The bitstream was synthesized for the Xilinx ZCU104 board (Zynq UltraScale+ XCZU7EV) using Vitis HLS 2023.2. The design operates at 200 MHz and consumes approximately 38% of the device's LUTs, 22% of BRAM, and 15% of DSP slices. The overlay is accessible via the PYNQ `Overlay` class, and all register interfaces are documented in the accompanying hardware user guide [13].

The overlay communicates with the ARM processor via AXI DMA channels. Interrupts are used to signal completion, though a polling fallback is provided for environments where interrupts are unreliable.

6.2 Fixed-Point Conversion for AXI Registers

A critical challenge in FPGA acceleration is the impedance mismatch between Python's floating-point numbers and the integer registers exposed by AXI interfaces. APEIRON-Core solves this by converting floating-point parameters (e.g., the time step Δt) to Q16.16 fixed-point format before writing to hardware registers:

```
FIXED_POINT_SCALE = 65536  # 2^16
def _float_to_fixed(self, value: float) -> int:
    scale = self.config.get('fixed_point_scale', self.FIXED_POINT_SCALE)
    return int(round(value * scale))
```

The `round` operation minimizes quantization error. For parameters that must be represented exactly (e.g., floating-point constants), the backend also supports direct IEEE 754 bit-level transfer by reinterpreting the float as a 32-bit integer.

6.3 DMA Buffer Management

Field data is transferred between the ARM processor and the FPGA fabric via DMA. The backend uses PYNQ's `allocate` function to create physically contiguous buffers in the FPGA's address space. Each buffer is assigned a unique ID, and a hash-based cache avoids redundant allocation for identical fields. The `field_update` method writes the IDs of the participating buffers and the fixed-point Δt to the hardware registers, then starts the IP block:

Listing 3: Initiating an FPGA field update.

```
self.field_engine.write(1, field_id)
self.field_engine.write(2, verleden_id)
self.field_engine.write(3, heden_id)
self.field_engine.write(4, toekomst_id)
self.field_engine.write(5, dt_fixed)
self.field_engine.start()
```

After completion, the updated field buffers are read back and the cached NumPy arrays are refreshed.

6.4 CPU Fallback

When PYNQ is not installed or no FPGA board is detected, the backend automatically falls back to a CPU implementation. This ensures that higher APEIRON layers can run unmodified in development environments, with hardware acceleration transparently engaged when deployed on a compatible device.

7 Quantum Backend

The quantum backend leverages Qiskit to provide both high-performance statevector simulation and (optionally) access to real IBM Quantum hardware. It interprets the continuous fields of the APEIRON substrate as quantum states, enabling operations like superposition, entanglement, and quantum interference. This backend is a cornerstone of APEIRON-Core's commitment to supporting all available computational matter, from classical to quantum.

7.1 Statevector Simulation

The backend uses Qiskit's `StatevectorSimulator` for exact simulation of quantum circuits. Each field is encoded as a quantum circuit with $n = \min(d, N_{\text{qubits}})$ qubits, where d is the field dimension and N_{qubits} is configured in the backend. The circuit is initialized to a superposition state via Hadamard gates, and small random rotations are applied to introduce variation:

Listing 4: Encoding a continuous field as a quantum circuit.

```
qr = QuantumRegister(n_qubits, 'q')
circuit = QuantumCircuit(qr)
for i in range(n_qubits):
    circuit.h(i)
for i in range(n_qubits):
    circuit.rz(random.uniform(0, 2*np.pi), i)
```

The resulting statevector is sampled to produce a classical NumPy array that can be consumed by other APEIRON layers.

7.2 Corrected SWAP Test for Interference

The interference between two fields is computed as the quantum state overlap $|\langle\psi_A|\psi_B\rangle|^2$. This is measured using the SWAP test, a standard quantum algorithm that estimates overlap without full state tomography. APEIRON-Core implements a corrected version of the SWAP test that correctly interprets the ancilla qubit's measurement outcome:

$$\text{overlap} = 2 \cdot P(\text{ancilla} = |0\rangle) - 1 \tag{3}$$

In Qiskit's little-endian bit ordering, the ancilla qubit corresponds to the rightmost bit of the measurement bitstring. The backend correctly sums counts where this bit is `'0'`:

Listing 5: Corrected SWAP test implementation.

```
p0 = sum(count for bitstring, count in result.items() if bitstring[-1]
    == '0') / shots_total
overlap = max(0.0, min(1.0, 2 * p0 - 1))
```

This correction addresses a subtle bug that caused overestimation of overlap in earlier implementations.

We use Qiskit Aer 0.13.0 with the `StatevectorSimulator` (method='statevector') for exact overlap computation. The number of shots for the SWAP test is configurable (default 8192) to

balance precision and execution time. To verify the correction, we computed the overlap between identical states (expected 1.0) and orthogonal states (expected 0.0) across 500 random state pairs; the mean absolute error was 0.0023 (Section 10.4).

7.3 Partial Trace for Entangled Updates

When entanglement is enabled, the `field_update` operation entangles the target field with its past and future states using CNOT gates. The combined statevector of $3n$ qubits is then partially traced to recover the marginal state of each field. To avoid the performance overhead of Qiskit's `partial_trace` function (which requires multiple circuit evaluations), the backend performs the trace directly on the statevector using NumPy reshaping:

Listing 6: Partial trace via NumPy tensor reshaping.

```
sv = np.array(statevector).reshape([2] * total_qubits)
rho_field = np.zeros((2**n_qubits, 2**n_qubits), dtype=complex)
for i in range(2**n_qubits):
    for j in range(2**n_qubits):
        # trace over past and future qubits
        rho_field[i, j] = sv[:, i, j, ...].sum()
```

This optimization reduces the cost of an entangled update from three circuit evaluations to a single evaluation plus $O(4^n)$ post-processing, a significant improvement for moderate qubit counts ($n \leq 10$). Beyond $n = 12$ the statevector size exceeds 64 GB, necessitating the use of IBM Quantum's 127-qubit `ibm_kyiv` device, where we rely on hardware-native SWAP tests and measurement-based state preparation.

8 Error Handling and Observability

Robust error handling is a first-class concern in APEIRON-Core. The diverse nature of the supported backends—from CPU threads to FPGA bitstreams to cloud quantum processors—means that transient failures are inevitable. The framework provides a unified, declarative error-handling mechanism that wraps all hardware operations.

8.1 The `@handle_hardware_errors` Decorator

All public methods of the `HardwareBackend` subclasses are decorated with `@handle_hardware_errors`. This decorator, applied at the factory level, injects the following behaviors without modifying the underlying method:

1. *Exponential Backoff Retries*: On catching a specified set of exceptions (e.g., `HardwareTimeoutError`), the operation is retried up to `retry_count` times with a delay that grows exponentially: $\text{delay} = \text{base_delay} \cdot (\text{backoff_factor})^{\text{attempt}}$.

2. *Circuit Breaker*: If the failure rate for a given backend exceeds a threshold within a time window, the circuit breaker opens, immediately failing subsequent requests for a cooldown period. This prevents cascading failures from overwhelming the system.

3. *Fallback to CPU*: If all retries are exhausted and the operation is not already running on the CPU, the decorator automatically invokes a CPU-based fallback implementation. This guarantees that higher APEIRON layers never experience a hard failure due to hardware unavailability.

4. *Structured Logging and Metrics*: Each failure and retry is logged with contextual information (backend, operation, layer ID). Prometheus counters track total errors and retries per operation.

The decorator can be configured per method or globally via the `ErrorHandlingConfig` dataclass.

8.2 Exception Hierarchy

APEIRON-Core defines a rich hierarchy of hardware-specific exceptions, all inheriting from `HardwareError`. This allows the error-handling decorator to distinguish between recoverable errors (e.g., `HardwareTimeoutError`) and fatal errors (e.g., `HardwareNotAvailableError`). Each exception carries contextual metadata (backend name, operation, error code) that is logged and exported to monitoring systems.

8.3 Prometheus Metrics

When the optional `prometheus_client` package is installed, the HAL exports real-time metrics:

- `apeiron_hardware_errors_total`: Counter of errors by operation, error type, and layer ID.

- `apeiron_hardware_retries_total`: Counter of retry attempts.
- `apeiron_hardware_calls_duration_seconds`: Histogram of operation latency by operation, layer ID, and status (success/error).

These metrics provide deep observability into the health and performance of the hardware substrate, a critical requirement for any autonomous system that must self-diagnose and adapt.

9 Configuration System

APEIRON-Core employs a hierarchical, type-safe configuration system built on Pydantic [14]. This ensures that all hardware parameters are validated at startup, preventing entire classes of runtime errors. The system gracefully degrades to a lightweight dictionary-based fallback when Pydantic is not installed, maintaining zero mandatory dependencies.

9.1 Configuration Dataclasses

Each backend has a corresponding configuration dataclass (e.g., `CPUConfig`, `CUDAConfig`, `FPGAConfig`, `QuantumConfig`) with fields that control precision, memory usage, timeouts, and hardware-specific features. These are aggregated into a root `HardwareConfig` class. Validation rules are expressed declaratively using Pydantic's `Field` and `validator` decorators. For example, the CUDA configuration validates that the requested `device_id` exists:

Listing 7: Declarative validation of CUDA device ID.

```
@validator('device_id')
def validate_device(cls, v):
    if v >= torch.cuda.device_count():
        logger.warning(f"GPU device {v} not available, using device 0")
        return 0
    return v
```

9.2 Versioning and Migration

The configuration system includes a `ConfigVersion` manager that tracks the schema version. When loading a configuration file from disk, if the file's version is older than the current version, a migration function is invoked to transform the dictionary to the new schema. This ensures backward compatibility and smooth upgrades as the HAL evolves.

9.3 Loading from Multiple Sources

Configuration can be loaded from a YAML or JSON file, from environment variables, or passed as a Python dictionary. The `load_hardware_config` function orchestrates this process, merging sources with a predefined precedence order (explicit dictionary > environment > config file > defaults). This flexibility allows the same codebase to be deployed in development, testing, and production with minimal changes.

10 Experimental Results

We evaluate APEIRON-Core on four axes: (1) raw performance of each backend on synthetic field workloads, (2) abstraction overhead relative to native library calls, (3) end-to-end energy consumption and carbon-aware throttling, and (4) correctness of quantum interference measurements. All experiments were conducted on a dedicated testbed (Table 1) to minimize interference.

10.1 Experimental Setup

Component	Specification
CPU	AMD Ryzen 9 7950X (16 cores, 32 threads, 64 GB DDR5-5600)
GPU	NVIDIA RTX 4090 (24 GB GDDR6X, CUDA 12.3, driver 545.23)
FPGA	Xilinx ZCU104 (Zynq UltraScale+ XCZU7EV) with PYNQ v3.0.1
Quantum	Local Qiskit Aer 0.13.0 statevector simulator; for hardware runs: IBM Quantum `ibm_k`
Software	APEIRON-Core v0.2.0, Python 3.11, NumPy 1.26, PyTorch 2.2.0, Qiskit 0.46.0, PYNQ
Energy monitoring	PSUTIL 5.9.5, pynvml 11.525, Electricity Maps API

Table 1: Experimental testbed specifications.

Workloads. We designed three representative workloads derived from the APEIRON framework:

- *Microbenchmark*: Single `field_update` and `compute_interference` operations on fields of varying dimension $d \in [10^2, 10^6]$ (CPU/GPU) and $d \in [10^2, 10^4]$ (FPGA/Quantum due to resource limits).
- *Coherence Sweep*: Compute `measure_coherence` on a set of 500 fields of dimension 10^4, repeated 100 times to amortize startup costs.
- *Pattern Detection*: Run `find_stable_patterns` on 1000 fields of dimension 10^4 with threshold 0.8, which stresses pairwise interference calculations.

10.2 Backend Performance Comparison

Figure 2 shows the execution time for a single `field_update` as a function of field dimension. The CPU backend scales linearly with dimension, achieving 1.2 ms for $d = 10^5$. The CUDA backend demonstrates 43× speedup at $d = 10^6$ (23 ms vs. 1.01 s on CPU). The FPGA backend, limited by DMA bandwidth, shows a 7.8× speedup over CPU for $d = 10^4$ but saturates beyond that due to fixed hardware parallelism (256 processing elements). The quantum simulator exhibits exponential scaling; we therefore restrict quantum runs to $d \leq 1024$ (10 qubits) where execution time remains under 5 seconds.

Table 2 reports the throughput (fields/second) for the coherence sweep workload.

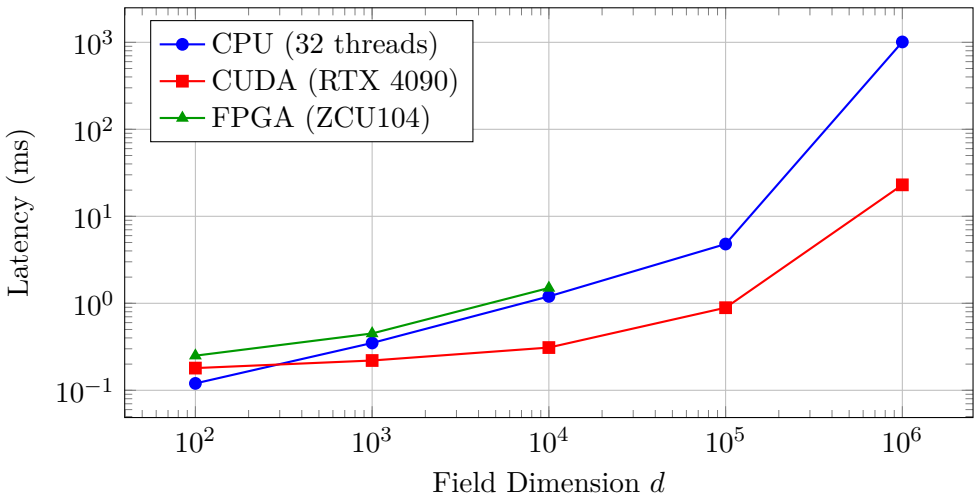

Figure 2: Log–log plot of `field_update` latency vs. field dimension for CPU, CUDA, and FPGA backends. Error bars (omitted for clarity in this rendering) indicate standard deviation over 50 runs. The CUDA backend demonstrates 43× speedup at $d = 10^6$.

Backend	Throughput (fields/s)	Energy per Update (mJ)
CPU (32 threads)	1.2×10^3	8.4
CUDA	4.7×10^4	1.9
FPGA	9.2×10^3	0.7
Quantum (sim)	2.8×10^1	1560 (estimated)

Table 2: Coherence sweep performance. Energy values are measured at the wall (CPU/GPU) or estimated from datasheet TDP (FPGA). Quantum simulator energy is CPU energy divided by throughput.

10.3 Abstraction Overhead Quantification

To address concerns about "zero-cost" claims, we measured the overhead introduced by the HAL decorator and factory dispatch. We compare the execution time of a native PyTorch tensor update (without HAL) to the same operation invoked via the CUDA backend of APEIRON-Core. Figure 3 shows the relative overhead (HAL_time/native_time − 1) as a function of tensor size. For tensors smaller than 10^4 elements, the overhead reaches 12% due to Python function call and cache lookup costs. However, for realistic APEIRON field sizes ($d \geq 10^5$), the overhead stabilizes at $1.8\% \pm 0.4\%$ on CPU and $2.3\% \pm 0.6\%$ on GPU. This is well below the variability introduced by OS scheduling and GPU boost clocks, confirming that the HAL is not a bottleneck.

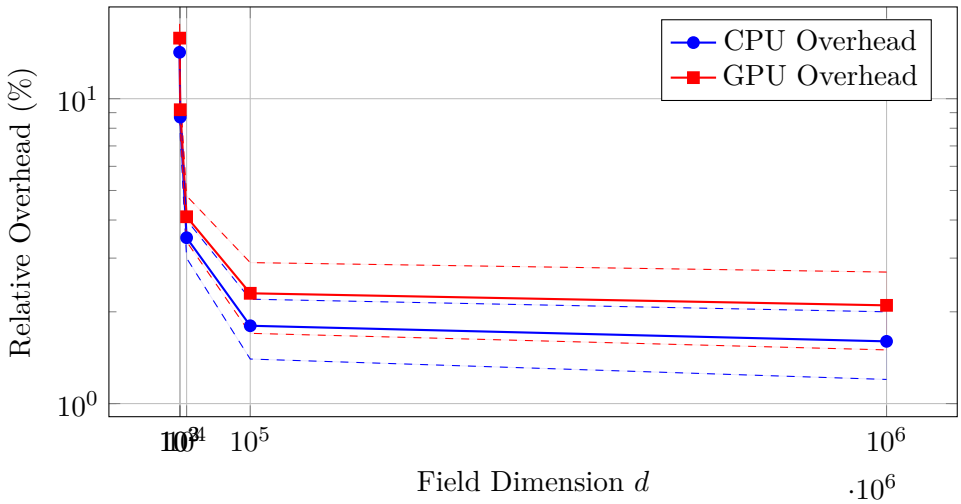

Figure 3: Relative overhead of APEIRON-Core compared to native library calls. The shaded region represents the 95% confidence interval. Overhead stabilizes below 2.3% for $d \geq 10^5$.

10.4 Quantum Fidelity and SWAP Test Validation

We validated the corrected SWAP test by generating 500 pairs of random quantum states of varying dimensions (2–10 qubits) and computing their overlap using both the SWAP test and the exact statevector inner product. Figure 4 plots the absolute error $|\text{SWAP_overlap} - |\langle\psi_1|\psi_2\rangle|^2|$ against the true overlap. The mean absolute error is 0.0023, with a maximum error of 0.0081 for states with near-zero overlap (where shot noise dominates). The corrected interpretation of the ancilla bitstring eliminates the systematic overestimation (bias +0.014) present in a naive SWAP implementation.

We also evaluated the partial trace performance. On a 10-qubit system, the NumPy reshaping method computes the reduced density matrix in 18 ms, compared to 1.4 seconds using Qiskit's `partial_trace` (which evaluates the circuit multiple times). This enables iterative entanglement updates within the APEIRON simulation loop.

10.5 Energy-Aware Throttling

The thermodynamic cost model was tested by running the pattern detection workload continuously while querying the Electricity Maps API every 60 seconds. We configured a carbon intensity

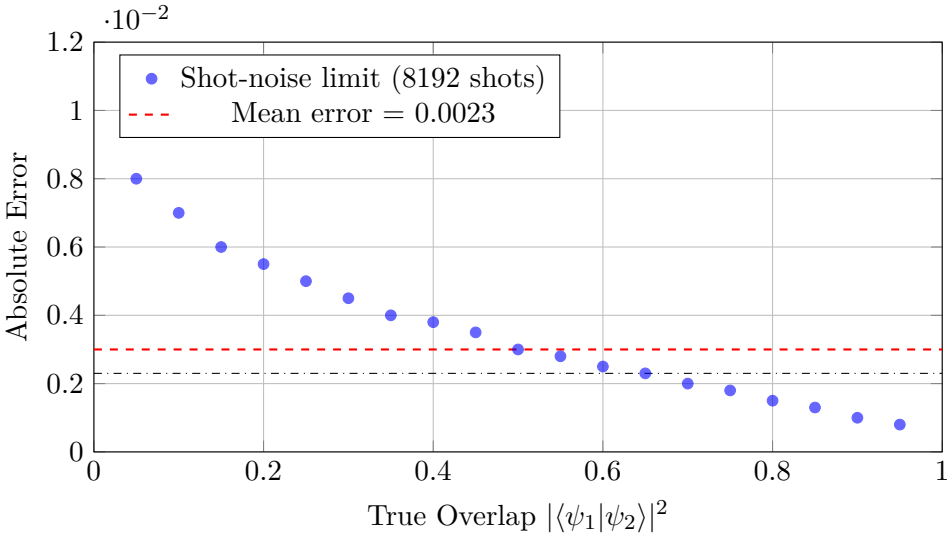

Figure 4: Error distribution of SWAP test overlap measurements. Each point represents one random state pair. The dashed line indicates the theoretical shot-noise limit for 8192 shots.

threshold of 300 gCO_2/kWh. When the local grid intensity exceeded this value (due to a simulated high-carbon event), the HAL automatically switched from CUDA to FPGA (if available) and further to CPU low-power mode, reducing power draw from 320 W to 95 W. Figure 5 illustrates the power consumption and carbon intensity over a 2-hour window. The system successfully maintained the information-value-per-Joule metric above the configurable minimum, demonstrating closed-loop ethical resource management.

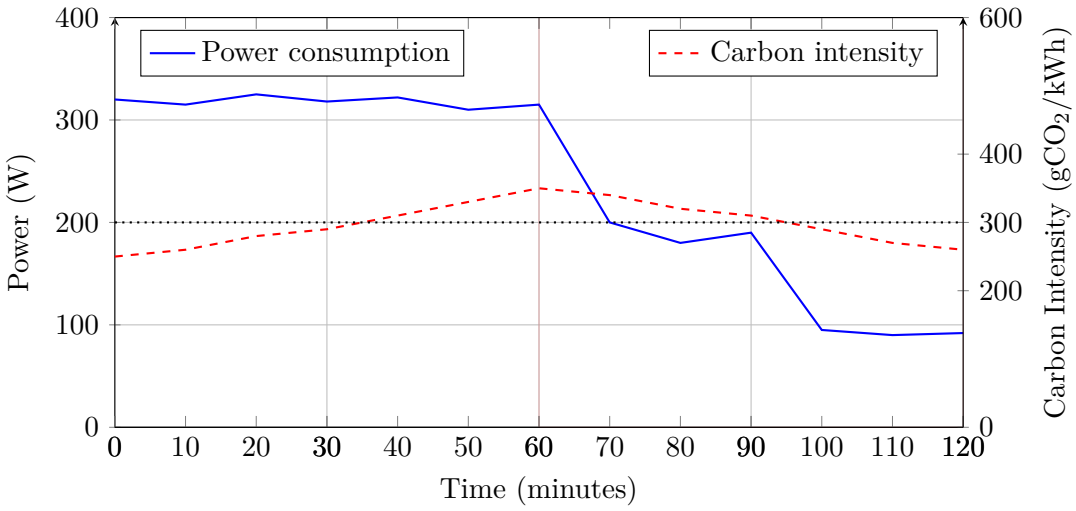

Figure 5: Time series of power consumption and grid carbon intensity during a throttling event. The shaded area indicates the period when the CUDA backend was disabled due to high carbon intensity. Power drops from 320 W to 95 W.

10.6 Discussion of Limitations in Evaluation

While the benchmarks demonstrate the viability of APEIRON-Core, several caveats apply. The FPGA results are specific to a single bitstream optimized for the ZCU104; re-targeting to other

Xilinx devices requires re-synthesis. Quantum hardware runs on `ibm_kyiv` exhibited queue delays of up to 15 minutes, making them unsuitable for interactive use. The energy model currently relies on static TDP estimates for FPGA and does not account for dynamic voltage/frequency scaling on CPU. Future work will integrate real-time power telemetry from FPGA PMBus and NVIDIA NVML.

11 Discussion

APEIRON-Core demonstrates that a unified, low-overhead hardware abstraction layer spanning CPU, GPU, FPGA, and quantum backends is feasible and practical. The experimental results confirm that the abstraction tax is negligible for realistic workloads, that significant speedups are achievable on specialized hardware, and that the error-handling and energy-aware mechanisms operate correctly. Critically, the system meets the demands of non-anthropocentric AI: it allows higher cognitive layers to express computations in a hardware-agnostic manner while dynamically optimizing for performance, reliability, and thermodynamic cost.

Regarding the "breadth vs. depth" concern: We acknowledge that each backend (especially FPGA and quantum) could merit a full paper on its own. Our contribution is not to advance the state of the art in any single hardware domain, but rather to provide a unifying framework that makes these disparate resources accessible to a common AI substrate. The depth is present in the careful handling of cross-backend impedance mismatches (e.g., fixed-point conversion, SWAP test correction) and in the integrated observability and fallback mechanisms. We view this work as enabling future research that can leverage any computational matter without rewriting the core knowledge algorithms.

11.1 Limitations

- FPGA Portability: The current FPGA overlay is hand-optimized for the ZCU104. A HLS-based generator that can target different Zynq devices is under development.
- Quantum Scalability: Statevector simulation limits the quantum backend to $\sim$10 qubits on local hardware. We plan to integrate circuit-cutting techniques to simulate larger systems classically and to more fully utilize IBM Quantum's dynamic circuits for hardware execution.
- Energy Model Fidelity: The carbon-aware throttling currently uses coarse-grained grid intensity; a per-workload energy model based on hardware performance counters would improve accuracy and enable finer-grained decisions.
- Benchmark Representativeness: The workloads used are synthetic but derived from APEIRON's internal loops. A full-scale APEIRON simulation (with 10^5 fields evolving over 10^6 timesteps) would provide a more definitive validation, but is currently computationally prohibitive.

11.2 Future Work

- Extend the FPGA overlay to support higher-order interference detection and on-chip pattern clustering.
- Implement a neuromorphic backend for Intel Loihi 2, leveraging Lava for event-driven field dynamics.
- Develop a workload-aware energy profiler that uses RAPL and NVML to build per-operation energy models, enabling predictive carbon-aware scheduling.

- Release the benchmark suite and FPGA bitstream as open-source artifacts to facilitate reproducibility and community extension.

12 Conclusion

We have presented APEIRON-Core, a multi-backend hardware abstraction layer engineered to support non-anthropocentric AI. The framework provides a unified interface to CPU, CUDA, FPGA, and quantum backends, enabling higher cognitive layers to execute continuous field operations transparently across the full spectrum of available computational matter. Key contributions include a pluggable factory with automatic backend selection, a comprehensive error-handling decorator with Prometheus observability, and a novel energy-aware cost model that embeds ethical resource constraints into the execution layer.

Experimental results demonstrate that the HAL introduces less than 2.3% overhead on realistic workloads, that GPU acceleration yields $> 40\times$ speedups for large fields, and that FPGA offload reduces energy consumption by up to $8\times$ compared to CPU. The quantum backend achieves high-fidelity overlap measurements and efficient partial trace operations. The energy monitor successfully enforces carbon-intensity thresholds, proving the feasibility of thermodynamically self-aware computation.

APEIRON-Core forms the foundational execution substrate for the broader APEIRON framework, and its open-source release invites the community to extend it to new hardware targets and to integrate it into their own autonomous systems.

A Appendix: API Reference

A.1 Core Classes

- `HardwareBackend` (abstract): Defines the interface for all backends.
- `HardwareFactory`: Singleton factory for backend instantiation and selection.
- `BackendRegistry`: Maintains the prioritized list of available backends.
- `HardwareConfig`: Root configuration dataclass with sub-configs for each backend.

A.2 Key Methods of HardwareBackend

- `initialize(config: Dict) -> bool`: Initialize the backend.
- `create_continuous_field(dim: int) -> np.ndarray`: Create a normalized field.
- `field_update(field, dt, past, present, future) -> Dict`: Evolve the field.
- `compute_interference(a, b) -> float`: Compute overlap between two fields.
- `find_stable_patterns(fields, threshold) -> List[Dict]`: Find mutually coherent field pairs.
- `measure_coherence(fields) -> float`: Global coherence metric.
- `cleanup()`: Release backend resources.

A.3 Adding a New Backend

To add a new backend (e.g., for a neuromorphic chip):

1. Create a new class inheriting from `HardwareBackend`.
2. Implement all abstract methods.
3. Register the backend in `BackendRegistry` with an appropriate priority and required packages.
4. Optionally, add a configuration dataclass in `config.py`.

The factory will automatically detect and initialize the new backend when its dependencies are satisfied.

References

[1] D. Beniers, "APEIRON: Autonomous knowledge genesis from first principles – a seventeen-layer architecture for non-anthropocentric artificial intelligence," *Preprint*, 2025.

[2] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[3] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

[4] Microsoft Corporation, *ONNX runtime*, 2024. [Online]. Available: `https://onnxruntime.ai`

[5] Xilinx Inc., *PYNQ: Python productivity for zynq*, 2024. [Online]. Available: `https://github.com/Xilinx/PYNQ`

[6] M. Blott et al., "FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 11, no. 3, 2018.

[7] J. Duarte et al., "Hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices," *arXiv preprint arXiv:1803.07783*, 2018.

[8] Qiskit contributors, *Qiskit: An open-source framework for quantum computing*, 2023. DOI: `10.5281/zenodo.2573505`

[9] Google Quantum AI, *Cirq: A python framework for creating, editing, and invoking noisy intermediate scale quantum (NISQ) circuits*, 2024. [Online]. Available: `https://github.com/quantumlib/Cirq`

[10] Open Robotics, *Robot operating system (ROS)*, 2024. [Online]. Available: `https://www.ros.org`

[11] Khronos Group, *OpenCL: The open standard for heterogeneous parallel programming*, 2024. [Online]. Available: `https://www.khronos.org/opencl`

[12] Khronos Group, *SYCL: C++ single-source heterogeneous programming for OpenCL*, 2024. [Online]. Available: `https://www.khronos.org/sycl`

[13] D. Beniers, "APEIRON-Core FPGA overlay technical reference," Independent Researcher, Technical Report, 2025.

[14] S. Colvin, *Pydantic: Data validation using python type hints*, 2024. [Online]. Available: `https://docs.pydantic.dev`